Symmetric ciphers and oracle attacks

Laboratory session 1

Nicola Laurenti, Simone Ragusa

April 1, 2026

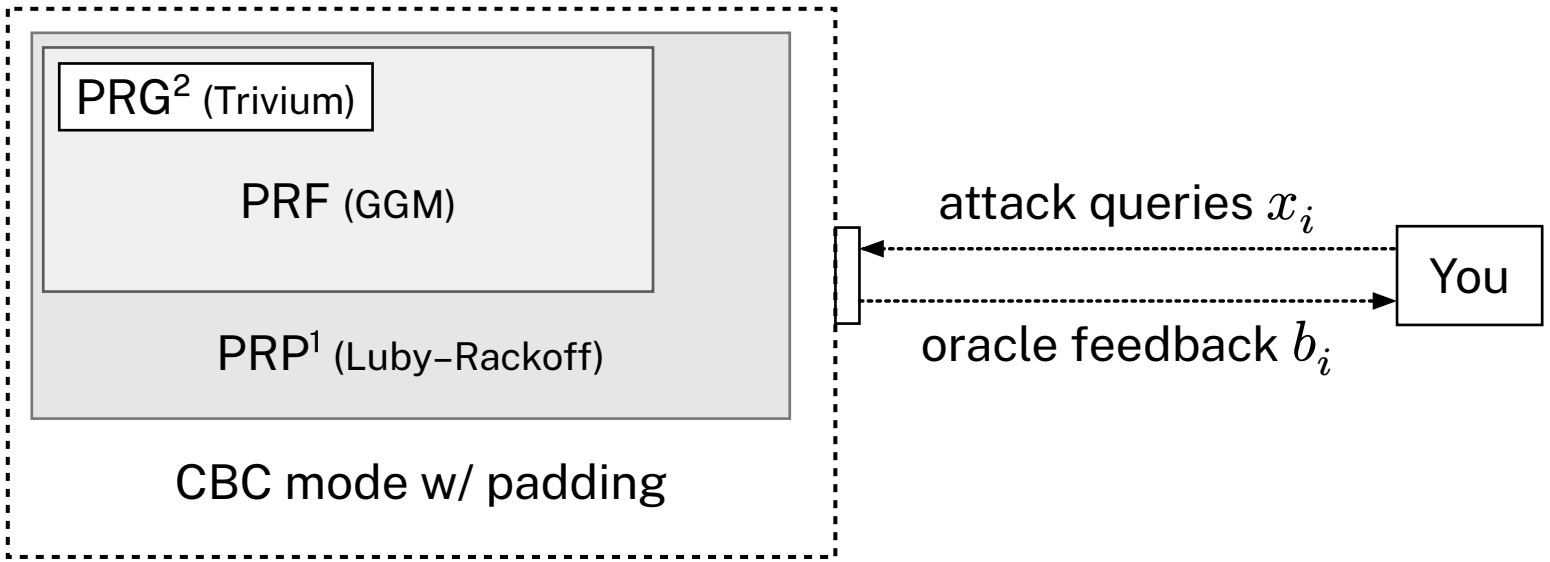
Except where otherwise stated, this work is licensed under
Creative Commons Attribution-ShareAlike 4.0 International.

Contents

Introduction to constructions and attacks 2

Tasks 18

Overall structure



[1] i.e., a block cipher

[2] e.g., key stream from stream cipher using the seed as the key and an all-zero initialization vector

Trivium

[De Cannière–Preneel, 2005]

Trivium is a hardware-oriented stream cipher was selected by the eSTREAM project, which ran from 2004 to 2008 to identify new stream ciphers for widespread adoption.

Parameters

- Key size: $\ell_k = 80 \text{ bit}$
- Initialization vector size: $\ell_{\text{iv}} = 80 \text{ bit}$
- Internal state size: $\ell_s = 288 \text{ bit}$

Setup

$$(s_1, s_2, \ldots, s_{93}) := (k_{80}, \ldots, k_1, 0, \ldots, 0)$$
$$(s_{94}, s_{95}, \ldots, s_{177}) := (\text{iv}_{80}, \ldots, \text{iv}_1, 0, \ldots, 0)$$
$$(s_{178}, s_{179}, \ldots, s_{288}) := (0, \ldots, 0, 1, 1, 1)$$

Warm-up

The state is rotated over 4 full cycles, in the same way as for key stream generation, but without generating key stream bits. This results in a total of 1152 rotations.

Trivium

[De Cannière–Preneel, 2005]

Key stream generation

$\underline{\textsc{NextKeyStreamBit}(s)}$:

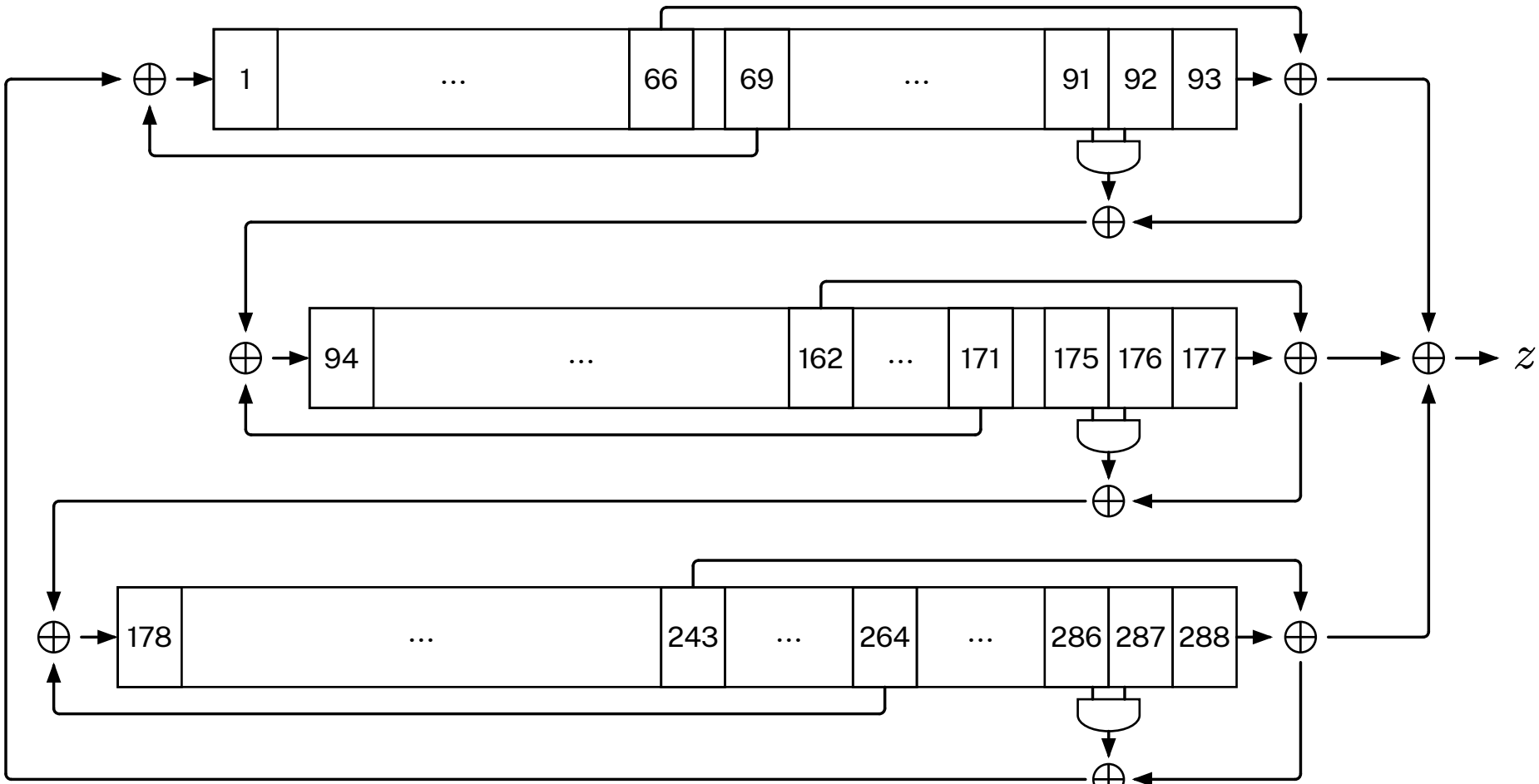
$t_1 := s_{66} + s_{93}$
$t_2 := s_{162} + s_{177}$
$t_3 := s_{243} + s_{288}$
$z := t_1 + t_2 + t_3$
$t_1 := t_1 + (s_{91} \cdot s_{92}) + s_{171}$
$t_2 := t_2 + (s_{175} \cdot s_{176}) + s_{264}$
$t_3 := t_3 + (s_{286} \cdot s_{287}) + s_{69}$
$(s_1, s_2, \ldots, s_{93}) := (t_3, s_1, \ldots, s_{92})$
$(s_{94}, s_{95}, \ldots, s_{177}) := (t_1, s_{94}, \ldots, s_{176})$
$(s_{178}, s_{179}, \ldots, s_{288}) := (t_2, s_{178}, \ldots, s_{287})$
return z

Remark

All operations are performed in the binary field $\mathrm{GF}(2)$. i.e., "$+$" is the XOR operation and "$\cdot$" corresponds to the AND operation.

Trivium

[De Cannière–Preneel, 2005]

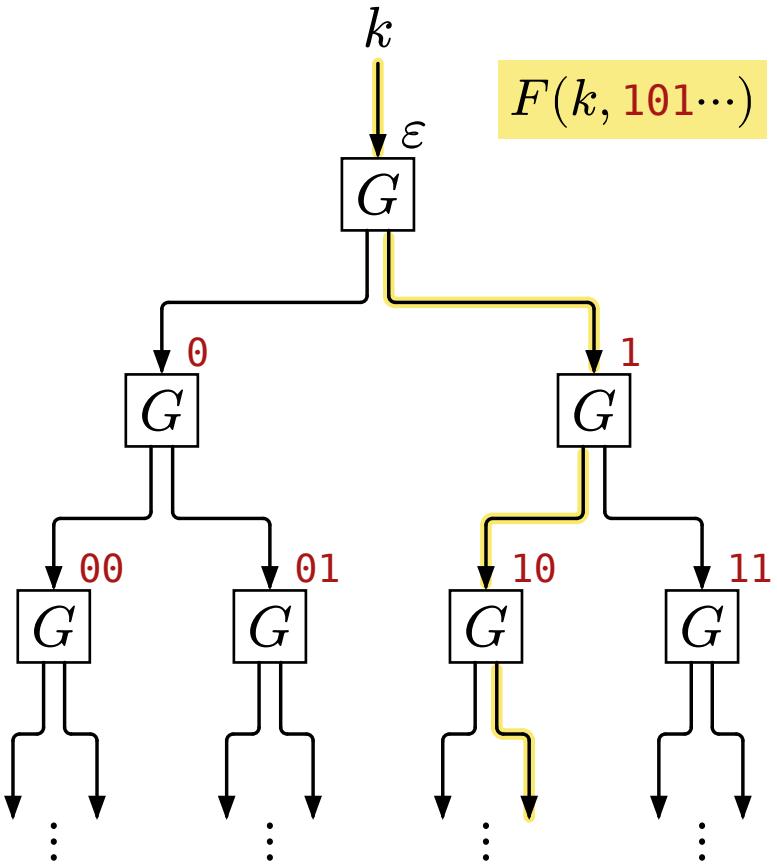


Constructing a PRF from a PRG

Construction (GGM) [Goldreich–Goldwasser–Micali, '84]

Let $G : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$ be a length-doubling pseudorandom generator. Then the GGM tree construction is defined as the following function $F : \{0,1\}^\lambda \times \{0,1\}^n \to \{0,1\}^\lambda$:

$\underline{F(k,u)}$:
  $s := k$
  for $i = 1$ to n:
    $r_0 \| r_1 := G(s)$
    $s := r_{u_i}$
  return s



Constructing a PRF from a PRG

Theorem

If G is a secure length-doubling PRG, then the GGM construction is a secure PRF.

Proof (omitted)

See *Introduction to Modern Cryptography, Proof of Theorem 8.21*.[1]

[1] Alternative reference: *A Graduate Course in Applied Cryptography, Proof of Theorem 4.10*.

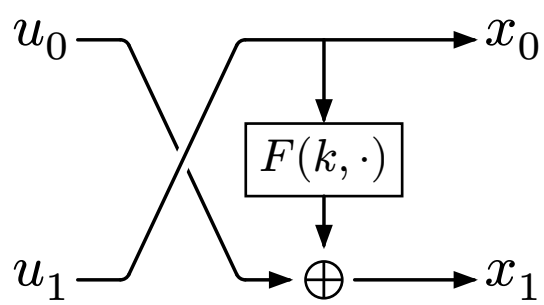
Constructing a block cipher from a PRF

Construction (keyed Feistel round)

Let $F : \{0,1\}^\lambda \times \{0,1\}^n \to \{0,1\}^n$ be a not-necessarily-invertible keyed function, which we call the round function. Then the keyed Feistel round is defined as the following invertible function $F^{(1)} : \{0,1\}^\lambda \times \{0,1\}^{2n} \to \{0,1\}^{2n}$:

$\underline{F^{(1)}(k, u_0 \| u_1):}$

return $u_1 \| (F(k, u_1) \oplus u_0)$



We call **Feistel cipher** (or Luby–Rackoff block cipher) the construction $F^{(r)}$ resulting from the successive application of $r \geq 3$ keyed Feistel rounds.

Constructing a block cipher from a PRF

Theorem (Luby–Rackoff)

If the round function F is a secure PRF, and the block size $2n$ is sufficiently large, then:

- the 3-round Feistel cipher is a secure pseudorandom permutation (aka block cipher);
- the 4-round Feistel cipher is a secure strong pseudorandom permutation (aka strong block cipher).

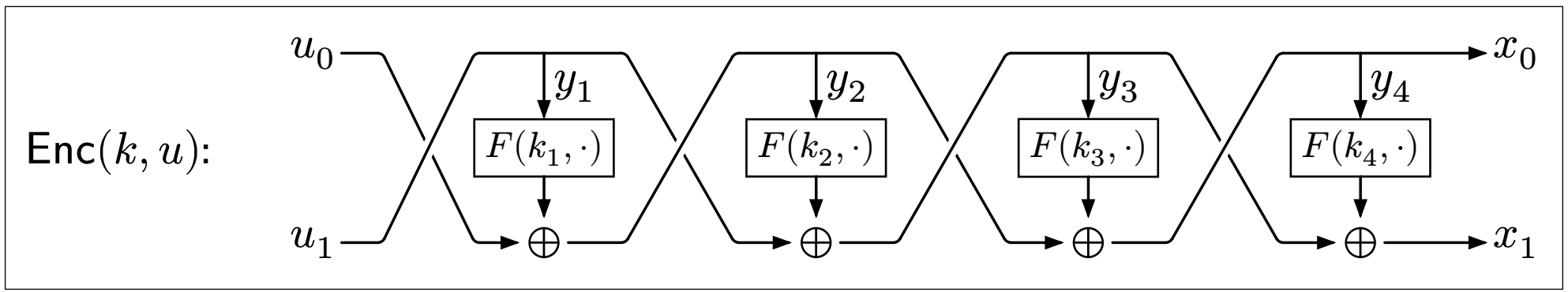
Proof (omitted)

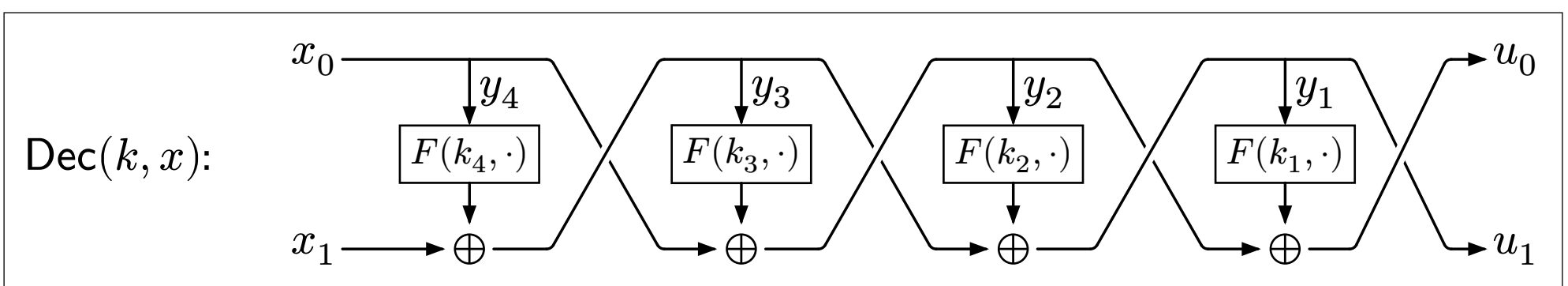
See *Introduction to Modern Cryptography, Proof of Theorem 8.22*.[1]

[1] Alternative reference: *A Graduate Course in Applied Cryptography, Proof of Theorem 4.9*.

Constructing a block cipher from a PRF

4-round Feistel cipher

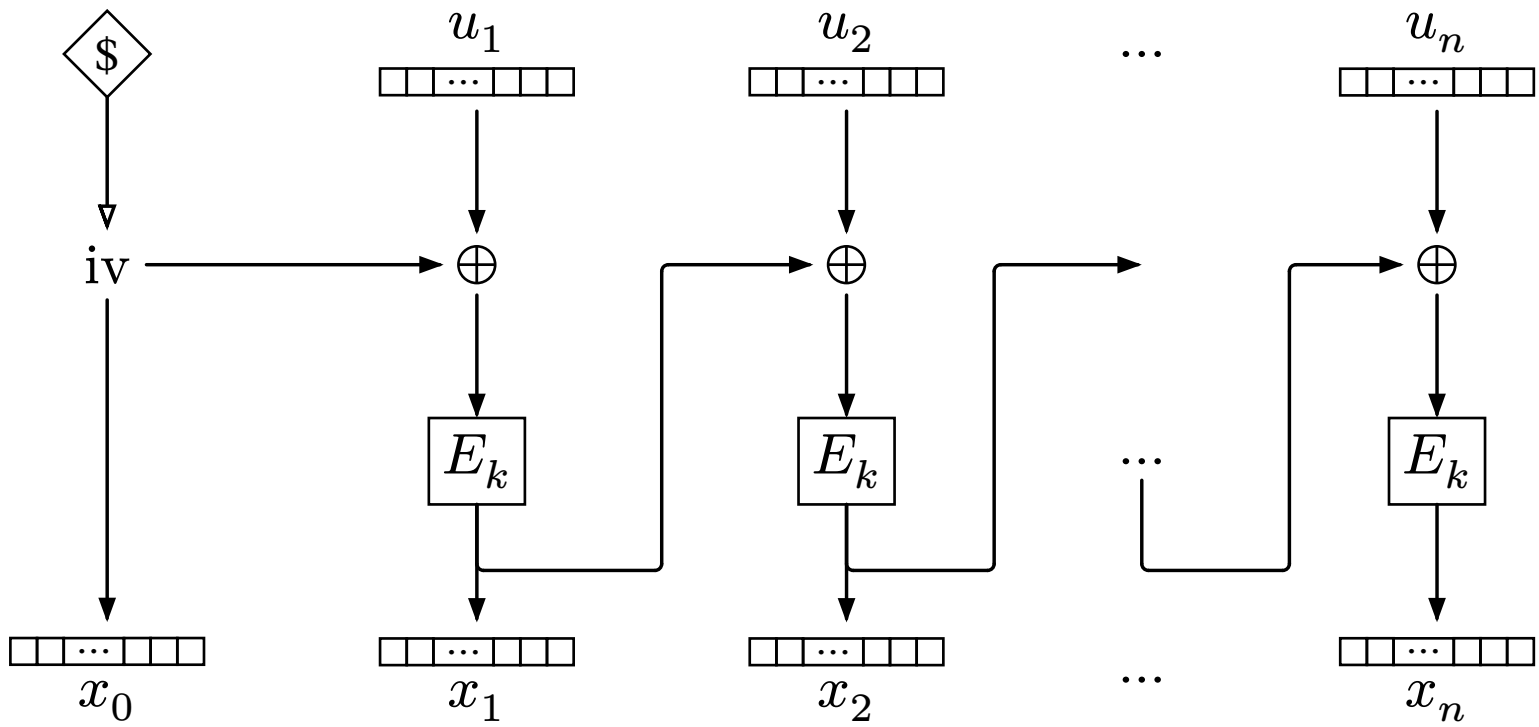




CBC mode of operation

Encryption of n-block plaintext u using initialization vector iv chosen uniformly at random from $\{0, 1\}^\ell$, where ℓ is the block size, and key k.

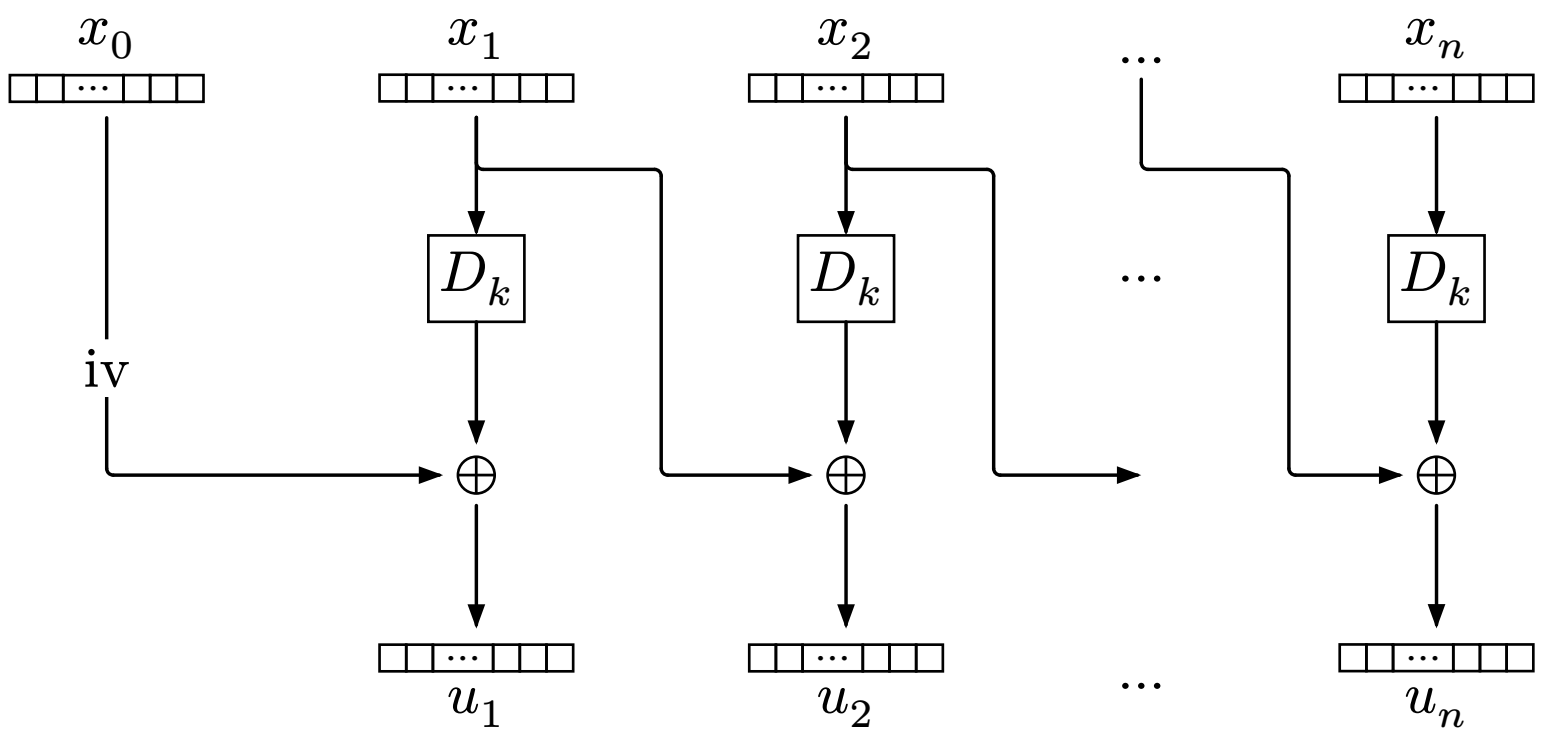
Enc-CBC(k, u):



CBC mode of operation

Decryption of $(n+1)$-block ciphertext x, with the initialization vector iv prepended as x_0, using key k.

Dec-CBC(k, x):



Padding

PKCS#7

If d bytes are needed to fill the last block, then append d copies of byte $\boxed{d}$ to the original plaintext. If the length of the original plaintext is an integer multiple of the block size ℓ, then an extra block of bytes with value $\boxed{\ell}$ is added.

Examples (PKCS#7 standard)

Let $\ell = 12 \text{ byte}$.

d8	32	9c	38	87	90	f1	70	a5	**03**	**03**	**03**	$\Rightarrow$ valid
d7	44	ca	9c	71	53	c0	1b	e0	47	65	**01**	$\Rightarrow$ valid
53	c1	81	a4	31	ba	0d	fa	c4	b6	f6	**5e**	$\Rightarrow$ invalid
c6	98	a1	05	de	40	e5	**05**	**05**	**05**	**3e**	**05**	$\Rightarrow$ invalid

Padding

ISO/IEC 7816-4 (aka “one and zeros”)

If d bytes are needed to fill the last block, then append a `80` byte followed by $d-1$ copies of the null byte `00` to the original plaintext. If the length of the original plaintext is an integer multiple of the block size ℓ, then an extra block with value `80 00 00 ... 00`.

Examples (ISO/IEC 7816-4 standard)

Let $\ell = 12$ byte.

c3	93	06	6a	7f	f1	54	d9	**80**	**00**	**00**	**00**

$\Rightarrow$ valid

15	3c	9c	27	69	26	58	ab	6d	2f	6c	**80**

$\Rightarrow$ valid

62	23	a8	88	52	e8	c4	f6	28	21	33	**4b**

$\Rightarrow$ invalid

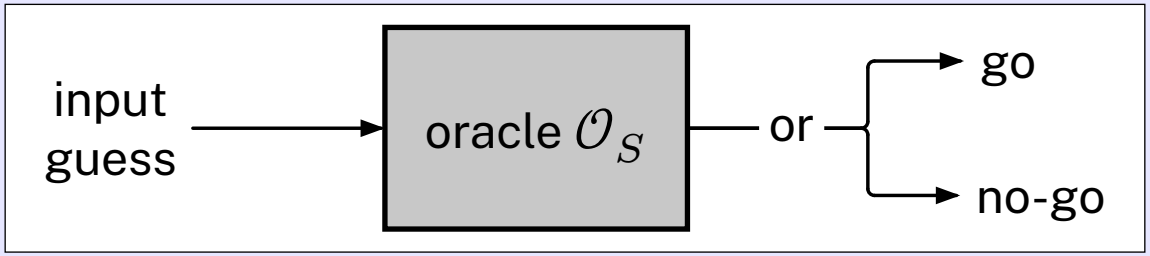
59	63	d9	5c	72	5a	c2	b2	c5	**33**	**00**	**00**

$\Rightarrow$ invalid

Oracle attacks

Definition (oracle attack, informal)

An oracle attack is any attack that aims to violate a security goal targeted by a system through the exploitation of a weakness resulting in a useful go/no-go indication.



Padding oracle attacks

A system that decrypts arbitrary inputs and reveals padding validation errors provides a decryption oracle (and, under certain conditions, an encryption oracle as well).

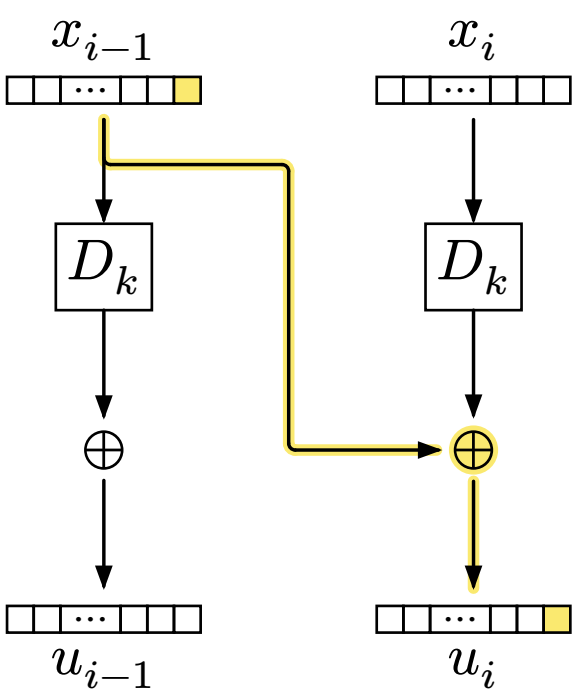
Oracle attacks

†assuming ISO/IEC 7816-4

Padding decryption oracle attack

[Vaudenay, 2002]

Fundamental insight: if you corrupt a ciphertext and discover that it contains valid padding, you know exactly what that padding byte is.



	$\cdots$?? ?? t		*target block u_i with target byte* t
$\oplus$	$\cdots$ 00 00 b		*mask applied to x_{i-1}*
$=$	$\cdots$?? ?? 80		*valid padding*† $\Leftrightarrow t \oplus b = 80$
			$\Leftrightarrow t = b \oplus 80$

Difficulty (ambiguous padding)

If the second-to-last byte of u_i is 80, then a value of b such that $t \oplus b = 00$ also yields valid padding.†

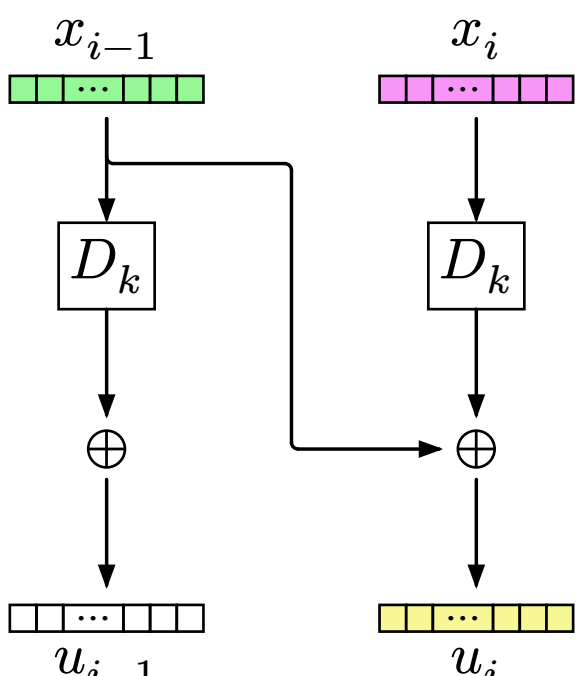
CC BY SA

Oracle attacks

Padding encryption oracle attack

[Rizzo–Duong, 2010]

Turn the decryption oracle into an encryption oracle by requesting random junk decryption and then constructing the ciphertext for the target plaintext backwards.



$\underline{\textsc{EncryptionOracle}\big(u \in \{0,1\}^{n\ell}\big)\text{:}}$

$x_n \twoheadleftarrow \{0,1\}^\ell$

for $i = n$ down to 1:

$\quad x_{i-1} := u_i \oplus D_k^{\text{oracle}}(x_i)$

return $x_0 \| x_1 \| \cdots \| x_n$ *// where x_0 is the IV*

Implementation guidelines

Byte-oriented interfaces

You should think of the functions you implement as either internal, carrying out intermediate operations, or public, callable by a third party (which may be future you).

Public functions performing cryptographic operations should always accept and return byte arrays only.

Test vectors

You will test your implementations using test vectors. All test vectors are provided as arrays of objects in JSON format.

Task 1 — Implement Trivium

Task 1

Using a programming language of your choice, implement the Trivium stream cipher. To convert bit-sequences to byte-sequences and vice versa, the first bit of the bit-sequence should be mapped to the least significant bit of the first byte of the byte-sequence. Verify the correctness of your implementation with the test vectors in `lab1task1.json`.

Test vector fields

- `number`: test vector progressive identifier
- `key`: 10-byte key represented as a hexadecimal string
- `iv`: 10-byte initialization vector represented as a hexadecimal string
- `stream`: expected first 32 bytes of the key stream represented as a hexadecimal string

Task 2 — Implement the GGM PRF

Task 2

Using a programming language of your choice, implement the GGM tree construction instantiated with Trivium as pseudorandom generator using the seed as the key and an all-zero initialization vector. Verify the correctness of your implementation with the test vectors in `lab1task2.json`.

Test vector fields

- `number`: test vector progressive identifier
- `key`: 10-byte key represented as a hexadecimal string
- `in`: {16,32}-byte PRF input represented as a hexadecimal string
- `out`: expected 10-byte PRF output represented as a hexadecimal string

Task 3 — Implement a Luby–Rackoff block cipher

Task 3

Using a programming language of your choice, implement a 4-round Luby–Rackoff block cipher according to the following parameters.

- Key size: $\ell_k = 40 \text{ byte}$
- Plaintext size: $\ell = 20 \text{ byte}$
- Round function: $F : \{\texttt{0}, \texttt{1}\}^{\ell/2} \times \{\texttt{0}, \texttt{1}\}^{\ell/2} \to \{\texttt{0}, \texttt{1}\}^{\ell/2}$ implemented as in Task 2
- Key schedule: split k as $k_1 \| k_2 \| k_3 \| k_4$ such that $|k_i| = \ell/2$, and use k_i at round i

Verify the correctness of your implementation with the test vectors in `lab1task3.json`, for both the encryption and decryption functions.

Task 3 — Implement a Luby–Rackoff block cipher

Test vector fields

- `number`: test vector progressive identifier
- `key`: 40-byte key represented as a hexadecimal string
- `msg`: 20-byte plaintext represented as a hexadecimal string
- `ct`: 20-byte ciphertext represented as a hexadecimal string

Task 4 — Break a 3-round Luby–Rackoff block cipher

Task 4

Consider a 3-round Feistel cipher $F^{(3)}$ using a secure PRF as round function. Devise a distinguisher between $F^{(3)}$ and an ideal random permutation and describe its program. The distinguisher has access to both the encryption and decryption oracles.

Hint

You may want to build a distinguisher for $F^{(2)}$ first to kickstart your reasoning (you will only need the encryption oracle there). You may also adapt your implementation from Task 3 to test your intuition and verify empirically the correctness of your distinguishers.

Task 5 — Implement the CBC mode of operation

Task 5

Using a programming language of your choice, implement the cipher block chaining mode of operation on top of the Luby–Rackoff block cipher from Task 3. The encryption function should accept variable-length plaintexts and apply ISO/IEC 7816-4 padding before encryption. Before returning the plaintext, the decryption function should validate the padding format and remove the padding.

Verify the correctness of your implementation with the test vectors in `lab1task5.json`, for both the encryption and decryption functions.

Task 5 — Implement the CBC mode of operation

Test vector fields

- `number`: test vector progressive identifier
- `key`: 40-byte key represented as a hexadecimal string
- `iv`: 20-byte initialization vector represented as a hexadecimal string
- `msg`: variable-length plaintext represented as a hexadecimal string
- `ct`: ciphertext represented as a hexadecimal string of length multiple of 20 bytes

Task 6 — Perform the padding decryption oracle attack

Task 6

Using a programming language of your choice, perform a padding (decryption) oracle attack against the following vulnerable endpoint that uses the cipher from Task 5.

```
https://interrato.dev/infosec/lab1?token=<your token goes here>
```

Your goal is to successfully decrypt the token assigned to your group in `lab1task6.json`.

Assignment vector fields

- `group`: name of the assigned group
- `token`: encrypted token represented as a hexadecimal string of length multiple of 20 bytes

Task 6 — Perform the padding decryption oracle attack

Hint

You may want to build a local padding oracle using your implementation from Task 5 to test your attack strategy locally, thus avoiding network latency and allowing you to iterate faster.

Caution

The vulnerable endpoint performs IP-based rate limiting to avoid accidental denial of service. You should retry any request that got a `429 Too Many Requests` status code as a response.

Task 7 — Perform the padding encryption oracle attack

Task 7

Using a programming language of your choice, perform a padding encryption oracle attack against the following vulnerable endpoint that uses the cipher from Task 5.

```
https://interrato.dev/infosec/lab1?token=<your token goes here>
```

Your goal is to craft an encrypted token that successfully grants you access to the privileged area. You should use the token you decrypted in Task 6 as a starting point.

Hint

As for the previous task, you may want to build a local padding encryption oracle using your implementation from Task 5 to test your attack strategy locally.

Task 8 — Discuss possible countermeasures

Task 8

Discuss and confront possible countermeasures to padding oracle attacks. Consider the trade-offs, generality, and prevention versus mitigation.

What you need to submit

Each group must turn in, through the Moodle assignment submission procedure:

1. the code of your implementations (well organized, as a compressed archive file);
2. a short report (1–3 pages) in PDF format, including:
   - 2.1. a brief description of your implementations for Tasks 1–3 and 5, explaining your choices and any challenges your encountered;
   - 2.2. the program of your distinguisher for Task 4 (either written in a programming language of your choice or as pseudocode), along with a brief explanation of how you developed it;
   - 2.3. the results of your attacks for Tasks 6 and 7, explaining your choices and any challenges your encountered;
   - 2.4. your discussion about countermeasures for Task 8.

Use of artificial intelligence tools must be adequately justified and disclosed with proper attribution. Group members are fully responsible for the submitted work.